

Kafka and Distributed Messaging

Session 2 of 7 | 4 Hours

Big Data Engineering Programme
Academic Year 2024–2025

Session Overview

Duration	4 hours (theory + lab)
Theory	Kafka internals: producers, consumers, durability
Lab	Build a Kafka streaming pipeline in Python
Prerequisites	Session 1 completed, Docker cluster running
Tools used	Python 3, kafka-python-ng, Docker Compose

🎯 Learning Objectives

By the end of this session, students will be able to:

- ✔ Explain Kafka's internal architecture: brokers, topics, partitions, offsets, and replication.
- ✔ Configure producers with the correct **acks**, **retries**, and batching parameters.
- ✔ Manage consumers with explicit offset control and consumer groups.
- ✔ Distinguish between *at-most-once*, *at-least-once*, and *exactly-once* delivery semantics.
- ✔ Write a complete Python producer-consumer pipeline using `kafka-python-ng`.
- ✔ Monitor pipeline health and consumer lag in Kafka UI.

Contents

1	Part I – Kafka Fundamentals: Brokers, Topics, Partitions	3
1.1	Recap: The Kafka Log	3
1.2	Anatomy of a Kafka Message	3
1.3	Topics and Partitions in Depth	4
1.4	Partition Routing: How Keys Determine Placement	4
2	Part II – Producers: Configuration and Guarantees	6
2.1	The Producer Lifecycle	6
2.2	The <code>acks</code> Parameter – Durability vs. Latency	6
2.3	Retries and Idempotence	6
2.4	Batching and Throughput Tuning	7
3	Part III – Consumers: Offsets, Groups, and Rebalancing	8
3.1	How Consumers Track Progress: Offsets	8
3.2	Offset Commit Strategies	8
3.2.1	Auto-commit (at-least-once, default)	8
3.2.2	Manual commit (precise control)	8
3.3	Consumer Groups and Partition Assignment	9
3.4	The Rebalance Protocol	9
3.5	Key Consumer Configuration Parameters	10
4	Part IV – Message Durability: The Full Picture	11
4.1	The Durability Triangle	11
4.2	ISR: In-Sync Replicas Explained	11
4.3	Log Compaction vs. Time-Based Retention	12
5	Part V – Lab Session	13
5.1	Step 0 – Environment Setup	13
5.1.1	0a – Start the Kafka Cluster	13
5.1.2	0b – Create a Virtual Environment and Install the Library	13
5.1.3	0c – Create the Topic for This Lab	14
5.2	Step 1 – Write the Producer	14
5.2.1	1a – Imports and Configuration	14
5.2.2	1b – Instantiate the Producer with Reliability Settings	14
5.2.3	1c – The Sending Loop	15
5.3	Step 2 – Write the Consumer	16
5.3.1	2a – Imports and Configuration	16
5.3.2	2b – Instantiate the Consumer	16
5.3.3	2c – The Processing Loop with Manual Commit	17
5.4	Step 3 – Run the Pipeline End-to-End	18
5.4.1	3a – Start the Consumer First	18
5.4.2	3b – Start the Producer	18
5.5	Step 4 – Consumer Group Scaling Exercise	19
5.5.1	4a – Start a Second Consumer Instance	19
5.5.2	4b – Check Partition Assignment	19

5.5.3	4c – Observe the Rebalance on Consumer Exit	19
5.6	Step 5 – Fault Injection: Crash and Replay	19
5.6.1	5a – Produce More Messages	19
5.6.2	5b – Crash the Consumer Mid-Processing	20
5.6.3	5c – Restart and Observe Replay	20
5.7	Step 6 – Monitor Consumer Lag in Kafka UI	20
6	Summary & Key Takeaways	22
6.1	Vocabulary Checklist	22
6.2	Further Reading	22
6.3	Preview: Session 3	23
Appendix A – Complete producer.py		24
Appendix B – Complete consumer.py		25
Appendix C – Kafka CLI Quick Reference		26
Appendix D – Glossary		27

1. Part I – Kafka Fundamentals: Brokers, Topics, Partitions

1.1 Recap: The Kafka Log

In Session 1 we set up a Kafka cluster and exchanged messages using the CLI. Before going further it is important to understand *exactly* how Kafka stores data.

Definition – The Kafka Commit Log

At its core, a Kafka partition is an **append-only, ordered, immutable log** stored on disk. Each new message is appended at the end and assigned a monotonically increasing integer called an **offset**. Messages are *never* modified or deleted individually; they expire according to the topic's retention policy.

This design is the source of many of Kafka's properties:

- **Sequential disk writes** are extremely fast on modern hardware (SSD and HDD).
- **Any consumer** can independently read at any offset, at any speed.
- **Replayability**: re-processing historical data requires only resetting an offset.
- **Immutability** eliminates complex concurrency control and simplifies recovery.

1.2 Anatomy of a Kafka Message

Every message (also called a *record*) that flows through Kafka has the following structure:

Table 1: Fields of a Kafka record

lightbg Field	Type	Role
Key	bytes (optional)	Determines partition routing; enables ordering per entity
Value	bytes	The actual payload (JSON, Avro, Protobuf, plain text...)
Timestamp	long (ms since epoch)	Event time or ingestion time, used for time-window operations
Headers	list of key/value	Optional metadata (tracing IDs, schema version, source system)
Partition	int	Assigned by producer or by Kafka after partitioning
Offset	long	Unique, sequential position within the partition

⚠ Common Pitfall

Kafka is **schema-agnostic** by default – it treats keys and values as raw bytes. It is the producer's responsibility to serialise data and the consumer's responsibility to deserialise it. Using inconsistent serialisers (e.g., producer uses JSON, consumer expects Avro) will produce silent data corruption. Always agree on a serialisation format at the topic level, and consider using a *Schema Registry* for production.

1.3 Topics and Partitions in Depth

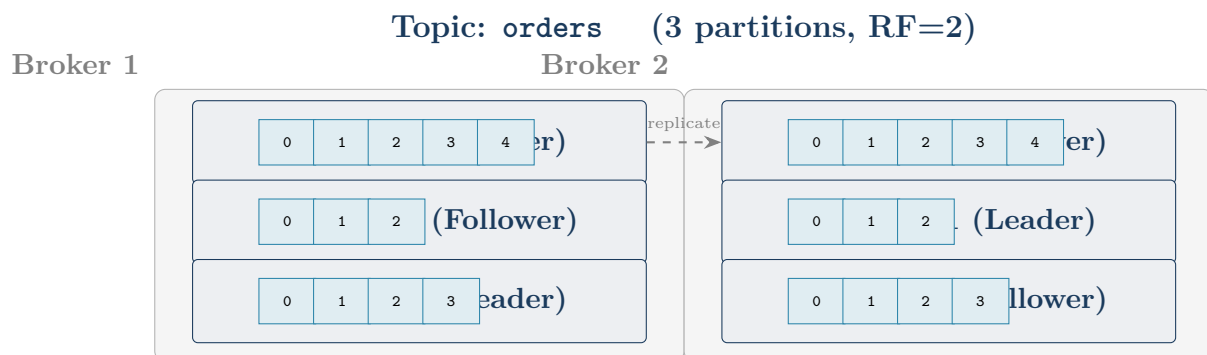


Figure 1: Topic **orders**: 3 partitions spread across 2 brokers (RF=2). Each partition has one leader and one follower. Messages are appended left-to-right within each partition, identified by their offset (0, 1, 2, ...).

1.4 Partition Routing: How Keys Determine Placement

When a producer sends a message, Kafka decides which partition to send it to using the following logic:

1. **Explicit partition** – producer specifies `partition=N` directly.
2. **Key-based routing** – Kafka computes `partition = hash(key) mod numPartitions` (default: *murmur2* hash). All messages with the same key always land on the same partition, preserving order for that key.
3. **Round-robin (no key)** – Kafka distributes messages evenly across all partitions for maximum throughput, sacrificing ordering.
4. **Custom partitioner** – the application provides its own `Partitioner` class implementing business logic.

! Choosing the Right Key

The partition key is one of the most impactful design decisions in a Kafka system:

- Use **entity IDs** (`user_id`, `order_id`, `sensor_id`) when ordering per entity matters and consumers process entities independently.
- Use **no key** (round-robin) when ordering does not matter and throughput is the priority.

- **Avoid low-cardinality keys** (e.g., country with only 3 values for 12 partitions)
 - they create *hot partitions* where one partition receives far more traffic than others.

2. Part II – Producers: Configuration and Guarantees

2.1 The Producer Lifecycle

A Kafka producer goes through four phases for each batch of messages:

1. **Serialise** – convert the key and value objects to bytes.
2. **Partition** – decide which partition to send to (Section 1).
3. **Accumulate** – buffer records in an in-memory batch per partition.
4. **Send** – flush the batch to the broker when either the batch is full (`batch.size`) or a timer expires (`linger.ms`).

2.2 The `acks` Parameter – Durability vs. Latency

The `acks` setting is the single most important producer configuration. It controls *how many broker acknowledgements the producer waits for before considering a message successfully sent*.

Table 2: Producer `acks` values compared

lightbg <code>acks</code>	Behaviour	Risk	Latency
0	Producer does not wait for any acknowledgement. Fire and forget.	Data loss if broker crashes before writing.	Lowest
1	Leader writes to its local log and acknowledges.	Data loss if leader crashes before followers replicate.	Medium
all (-1)	Leader waits until all ISR replicas have written the record.	No data loss as long as at least one ISR is alive.	Highest

Production Recommendation

Always use `acks=all` combined with `min.insync.replicas=2` for production pipelines. This ensures that even if one broker crashes during a write, the message is already safe on at least one other broker. The latency overhead is typically only a few milliseconds on a well-tuned cluster.

2.3 Retries and Idempotence

Network hiccups and broker leader elections can cause transient failures. The producer can automatically retry failed sends:

Table 3: Key producer reliability settings

lightbg Parameter	Default	Effect
<code>retries</code>	2147483647	Number of automatic retry attempts on transient errors
<code>retry.backoff.ms</code>	100ms	Wait time between retries (avoids hammering a failing broker)
<code>delivery.timeout.ms</code>	120 000ms	Total time budget for one send attempt including all retries
<code>enable.idempotence</code>	true	Assigns a sequence number to each record; broker deduplicates
<code>max.in.flight.requests</code>	5	Max unacknowledged requests per connection

Definition – Idempotent Producer

With `enable.idempotence=true`, the broker assigns each producer a **Producer ID (PID)** and tracks a **sequence number** per partition. If the producer retries after a transient failure, the broker recognises the duplicate sequence number and discards the duplicate – guaranteeing **exactly-once delivery at the partition level**.

2.4 Batching and Throughput Tuning

Kafka achieves high throughput by batching multiple records into a single network request. Two parameters control this trade-off:

- **batch.size** (default: 16 384 bytes) – maximum byte size of a batch per partition. Kafka sends the batch immediately once full.
- **linger.ms** (default: 0 ms) – how long the producer waits before sending a non-full batch. Setting this to 5–20 ms dramatically improves throughput at the cost of a small latency increase.

Throughput vs. Latency Trade-off

lightbg Goal	linger.ms	batch.size	compression
Max throughput	20–50	131 072	snappy / lz4
Min latency	0	16 384	none
Balanced	5–10	65 536	lz4

3. Part III – Consumers: Offsets, Groups, and Rebalancing

3.1 How Consumers Track Progress: Offsets

Unlike traditional message queues that delete a message once consumed, Kafka keeps all messages. The consumer is responsible for tracking its own position using **offsets**.

Definition – Consumer Offset

An **offset** is the position of the *next* message a consumer wants to read within a partition. Kafka stores committed offsets in an internal topic called `__consumer_offsets`. On restart, a consumer resumes from its last committed offset.

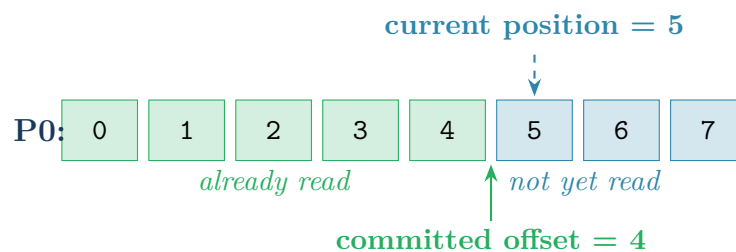


Figure 2: Partition P0: offsets 0–4 have been read, offset 4 has been committed. The consumer’s next fetch will start at offset 5.

3.2 Offset Commit Strategies

Kafka offers two commit modes, each with different delivery guarantees:

3.2.1 Auto-commit (at-least-once, default)

- Enabled with `enable.auto.commit=True` (default interval: `auto.commit.interval.ms=5000`).
- The Kafka client commits the offset automatically in a background thread.
- **Risk:** if the application crashes between processing a message and the auto-commit, the message will be reprocessed on restart.
- **Use when:** processing is idempotent (safe to repeat).

3.2.2 Manual commit (precise control)

- Set `enable.auto.commit=False`; call `consumer.commit()` yourself.
- **Commit after processing:** guarantees at-least-once delivery.
- **Commit before processing:** achieves at-most-once (may lose messages).
- **Transactional:** commit Kafka offset and database write atomically for exactly-once end-to-end semantics.

Table 4: Delivery semantics summary

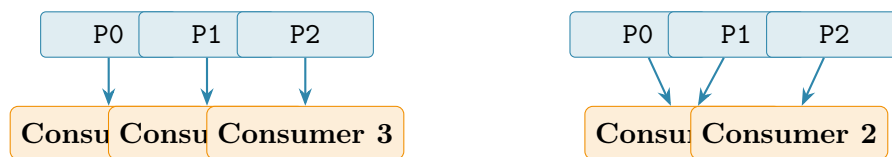
lightbg Semantic	How	Trade-off
At-most-once	Commit before processing	May lose messages; never duplicates
At-least-once	Commit after processing (default)	May duplicate; never loses messages
Exactly-once	Idempotent producer + transactional API	No loss, no duplicates; more complex

3.3 Consumer Groups and Partition Assignment

Definition – Consumer Group

A **consumer group** is a named set of consumer instances that cooperatively read from a topic. Kafka guarantees that each partition is assigned to *at most one* consumer in the group at any given time. This is how Kafka achieves **parallel, fault-tolerant consumption**.

(A) Ideal: 3 consumers, 3 partitions (B) 2 consumers, 3 partitions



Consumer 1 handles P0+P1; Consumer 2 handles P2. No partition is left unread.

Figure 3: Consumer group rebalancing ensures all partitions are always covered.

3.4 The Rebalance Protocol

A **rebalance** is triggered whenever the group membership changes: a consumer joins, leaves, crashes, or a topic gains new partitions.

1. All consumers are notified by the **Group Coordinator** (a broker).
2. Each consumer stops processing and *revokes* its current partitions.
3. A **Group Leader** (one consumer elected within the group) computes the new partition assignment.
4. The new assignment is distributed by the Group Coordinator.
5. Consumers resume processing from their committed offsets.

⚠ Common Pitfall

During a rebalance, **all consumption stops** (“stop-the-world” rebalance). For high-throughput applications, rebalances can cause significant lag spikes. Use `session.timeout.ms` and `heartbeat.interval.ms` carefully: too-short timeouts trigger unnecessary rebalances; too-long timeouts delay detection of crashed consumers. A good starting point is `session.timeout.ms=45000` and `heartbeat.interval.ms=15000`.

3.5 Key Consumer Configuration Parameters

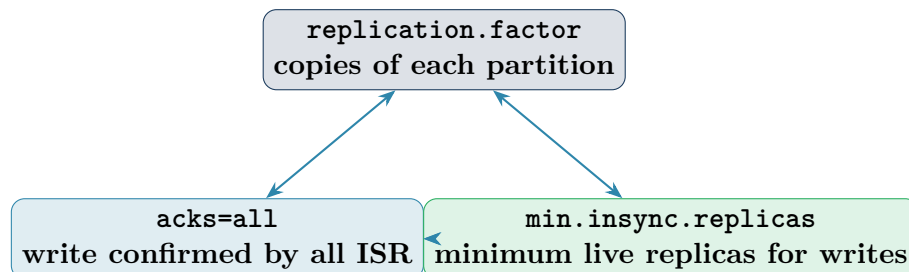
Table 5: Essential consumer parameters

lightbg Parameter	Default	Effect
<code>group.id</code>	(required)	Identifies the consumer group; consumers sharing a <code>group.id</code> cooperate
<code>auto.offset.reset</code>	latest	Where to start if no committed offset exists: earliest (replay all) or latest (new only)
<code>enable.auto.commit</code>	true	Automatically commit offsets every <code>auto.commit.interval.ms</code>
<code>max.poll.records</code>	500	Max records returned per <code>poll()</code> call – tune per processing speed
<code>fetch.min.bytes</code>	1	Min data to fetch before returning; increase for throughput
<code>session.timeout.ms</code>	45 000	Time before consumer is considered dead if no heartbeat is received
<code>heartbeat.interval.ms</code>	3 000	Frequency of heartbeat to the Group Coordinator

4. Part IV – Message Durability: The Full Picture

4.1 The Durability Triangle

Kafka's durability guarantees come from three cooperating mechanisms:



All three must be configured together for strong durability guarantees.

Figure 4: The durability triangle: replication factor, acks, and min.insync.replicas.

4.2 ISR: In-Sync Replicas Explained

Definition – In-Sync Replica (ISR)

The **ISR** is the set of partition replicas that are *fully caught up* with the leader. A replica is removed from the ISR if it falls more than `replica.lag.time.max.ms` (default: 30 000 ms) behind the leader. Only ISR members are eligible to become the new leader on failover.

The interaction between `acks=all` and `min.insync.replicas` is critical:

- `acks=all` means the leader waits for *all current ISR members* to acknowledge the write – but if ISR has shrunk to 1 (only the leader), `acks=all` offers no more protection than `acks=1`.
- `min.insync.replicas=2` adds a floor: if fewer than 2 replicas are in the ISR, the *broker* refuses the write entirely, throwing a `NotEnoughReplicasException`. This is the safety net.

Table 6: Durability scenarios with RF=3, min.insync.replicas=2

lightbg Brokers alive	ISR size	Result
3	3	Normal operation. Write succeeds; confirmed by 3 replicas.
2	2	Write succeeds; ISR at minimum. One more failure = outage.
1	1	Write fails with <code>NotEnoughReplicasException</code> . Data is protected.

4.3 Log Compaction vs. Time-Based Retention

Kafka offers two ways to manage how long data stays on disk:

Table 7: Retention strategies

Strategy	How it works	Use case
Time-based retention	Delete segments older than <code>retention.ms</code> (default: 7 days) or larger than <code>retention.bytes</code>	Event streams, logs, metrics
Log compaction	Keep only the <i>latest</i> record per unique key; older versions deleted in background	CDC (Change Data Capture), user profiles, config tables – maintaining current state

5. Part V – Lab Session

Lab 2 – Build a Kafka Streaming Pipeline in Python

Duration 90 minutes

Objective Write a Python producer and consumer using `kafka-python-ng`; configure acks, retries, and timeouts

Tools Python 3.9+, `kafka-python-ng` library, Docker cluster from Session 1

Difficulty ●● Beginner-Intermediate

Lab Timeline

Time	Activity	Goal
0–10 min	Setup: start cluster, install lib	Verify environment
10–30 min	Write and run the producer	Understand acks and batching
30–50 min	Write and run the consumer	Manual offset commit
50–65 min	Consumer group exercise	Observe partition assignment
65–80 min	Fault injection	Simulate crash, observe replay
80–90 min	Monitor lag in Kafka UI	End-to-end observability

5.1 Step 0 – Environment Setup

5.1.1 0a – Start the Kafka Cluster

If your cluster from Session 1 is not running, restart it:

```
cd kafka-lab
docker compose up -d
docker compose ps      # all containers should show "Up"
```

i The cluster from Session 1 is reused unchanged. If you deleted the volumes (`docker compose down -v`), any previously created topics are gone and will need to be re-created. Named volumes survive `docker compose down` (without `-v`).

5.1.2 0b – Create a Virtual Environment and Install the Library

```
python3 -m venv venv
source venv/bin/activate      # Linux / macOS
# venv\Scripts\activate      # Windows

pip install kafka-python-ng
```

Why a virtual environment? A `venv` isolates this project's dependencies from your system Python, preventing version conflicts with other projects. `kafka-python` is the most widely used pure-Python Kafka client; it supports producers, consumers, admin operations, and transactions without requiring a Java runtime.

5.1.3 0c – Create the Topic for This Lab

```
docker exec -it kafka1 bash

kafka-topics --bootstrap-server kafka1:29092 \
  --create \
  --topic sensor-events \
  --partitions 3 \
  --replication-factor 3

kafka-topics --bootstrap-server kafka1:29092 \
  --describe --topic sensor-events
```

We create a new topic **sensor-events** to simulate IoT temperature sensor readings. Using 3 partitions lets us assign one partition per sensor type (temperature, humidity, pressure) – all readings from the same sensor type will arrive in order within their partition.

5.2 Step 1 – Write the Producer

Create a new file `producer.py` in your project directory. We build it in three sub-steps.

5.2.1 1a – Imports and Configuration

```
from kafka import KafkaProducer
import json
import time
import random
```

KafkaProducer is the main class from `kafka-python-ng`. **json** is used to serialise our Python dictionaries into bytes. **time** and **random** are used to simulate sensor readings with delays.

5.2.2 1b – Instantiate the Producer with Reliability Settings

```
producer = KafkaProducer(
    bootstrap_servers=[ 'localhost:9092',
                        'localhost:9094',
                        'localhost:9096' ],

    acks='all',
    retries=5,
    max_in_flight_requests_per_connection=1,
    value_serializer=lambda v: json.dumps(v).encode('utf-8'),
    key_serializer=lambda k: k.encode('utf-8'),
    linger_ms=10,
    batch_size=32768,
)
```

bootstrap_servers: all three broker addresses are listed as a fallback. If the first broker is unreachable, the client tries the next one automatically. After the initial connection, Kafka provides the full cluster metadata.

acks='all': every send waits for confirmation from all ISR replicas. This is the strongest durability guarantee and protects against broker crashes.

Note – enable_idempotence: the `kafka-python-ng` library does **not** support the `enable_idempotence` parameter (raises `TypeError` if present). Use `acks='all' + retries=5` to get *at-least-once* delivery guarantees. True exactly-once semantics require the Confluent Kafka client or Kafka Streams.

max_in_flight_requests_per_connection=1: set to 1 to preserve message ordering during retries when `retries > 0`.

value_serializer / key_serializer: Kafka transports bytes, so we must convert Python objects to bytes before sending. Here we encode the value dict as a UTF-8 JSON string and encode the key string as UTF-8 bytes.

linger_ms=10: wait up to 10ms to fill a batch before sending. **batch_size=32768**: maximum batch size (32 KB). Together these two parameters significantly improve throughput.

5.2.3 1c – The Sending Loop

```
SENSOR_TYPES = ['temperature', 'humidity', 'pressure']
TOPIC = 'sensor-events'

def send_reading(sensor_type, value):
    record = {
        'sensor': sensor_type,
        'value': value,
        'unit': {'temperature': 'C',
                 'humidity': '%',
                 'pressure': 'hPa'}[sensor_type],
        'timestamp': int(time.time() * 1000),
    }
    future = producer.send(
        topic=TOPIC,
        key=sensor_type,          # key => same sensor always same
                                # partition
        value=record,
    )
    return future

try:
    for i in range(50):
        sensor = random.choice(SENSOR_TYPES)
        value = round(random.uniform(10, 40), 2)
        future = send_reading(sensor, value)
        # Block until broker confirms receipt
        metadata = future.get(timeout=10)
        print(f"Sent to partition {metadata.partition}, "
              f"offset {metadata.offset}")
        time.sleep(0.2)
finally:
    producer.flush()           # send any remaining buffered records
```

```
producer.close()
print("Producer closed.")
```

producer.send() is *asynchronous* – it returns a `FutureRecordMetadata` immediately without waiting.

future.get(timeout=10) blocks until the broker acknowledges the write (or raises an exception after 10 seconds). In production you would typically use callbacks instead of `.get()` to avoid blocking the main thread:

```
producer.send(TOPIC, key=k, value=v)\
    .add_callback(on_success)\
    .add_errback(on_error)
```

key=sensor_type means all readings from the same sensor type always land on the same partition. This guarantees that a downstream consumer processing **temperature** readings sees them in chronological order.

producer.flush() before closing drains the internal send buffer, ensuring all buffered records are sent before the process exits.

5.3 Step 2 – Write the Consumer

Create a new file `consumer.py`.

5.3.1 2a – Imports and Configuration

```
from kafka import KafkaConsumer, TopicPartition
import json
```

KafkaConsumer subscribes to topics and fetches records. **TopicPartition** is a named tuple used when manually assigning or seeking specific partitions – you will use it in Step 3.

5.3.2 2b – Instantiate the Consumer

```
consumer = KafkaConsumer(
    'sensor-events',
    bootstrap_servers=['localhost:9092',
                      'localhost:9094',
                      'localhost:9096'],
    group_id='sensor-analytics',
    auto_offset_reset='earliest',
    enable_auto_commit=False,
    value_deserializer=lambda b: json.loads(b.decode('utf-8')),
    key_deserializer=lambda b: b.decode('utf-8') if b else None,
    max_poll_records=10,
    session_timeout_ms=45000,
    heartbeat_interval_ms=15000,
)
```

group_id='sensor-analytics': registers this consumer in a named group. Multiple instances running with the same **group_id** will share partitions.

auto_offset_reset='earliest': when the group has no committed offset (first run), start from the very first message in each partition. Change to **'latest'** to process only new messages.

enable_auto_commit=False: we disable automatic commits so we control *exactly when* an offset is marked as processed. This prevents data loss when the consumer crashes mid-batch.

value_deserializer: the mirror image of the producer's **value_serializer** – converts bytes back to a Python dictionary.

max_poll_records=10: fetch at most 10 records per **poll()**. This limits the amount of work between commits, reducing reprocessing on crash.

5.3.3 2c – The Processing Loop with Manual Commit

```
print("Consumer started. Waiting for messages...")

try:
    for message in consumer:
        record = message.value
        key     = message.key
        part    = message.partition
        offset  = message.offset

        #           Business logic

        print(f"[P{part}|O{offset}] sensor={key} "
              f"value={record['value']} {record['unit']}")

        if record['sensor'] == 'temperature' \
           and record['value'] > 35:
            print(f"    >>> ALERT: high temperature "
                  f"{record['value']} C !")

        #           Commit AFTER processing

        consumer.commit()

except KeyboardInterrupt:
    print("Stopping consumer...")
finally:
    consumer.close()
    print("Consumer closed.")
```

for message in consumer: iterates over records as they arrive. Internally, this calls `poll()` in a loop and yields individual records.

consumer.commit() is called *after* the record is fully processed. This is the **at-least-once** pattern: if the application crashes between processing and committing, the message will be replayed on restart. As long as your processing logic is idempotent (e.g., writing to a database with an upsert), this is the safest approach for most applications.

commit() after each message is safe but slow. For higher throughput, commit after each *batch* (every N records) – see the advanced pattern below.

⚡ Advanced: Batch Commit Pattern

For better throughput, commit after every batch of records rather than every individual message. Use `poll()` directly instead of the `for` iterator:

```
BATCH_SIZE = 50

while True:
    records = consumer.poll(timeout_ms=1000,
                             max_records=BATCH_SIZE)

    if not records:
        continue
    for partition, msgs in records.items():
        for msg in msgs:
            process(msg)
    consumer.commit()    # commit once per batch
```

This reduces the number of commits from one per message to one per batch, significantly improving throughput. The trade-off: on crash, up to `BATCH_SIZE` messages may be reprocessed.

5.4 Step 3 – Run the Pipeline End-to-End

5.4.1 3a – Start the Consumer First

```
# Terminal 1
python consumer.py
```

Start the consumer *before* the producer so you can observe messages arriving in real time. The consumer will block on `poll()` until messages start flowing.

5.4.2 3b – Start the Producer

```
# Terminal 2
python producer.py
```

You should immediately see messages appearing in Terminal 1. Observe:

- All `temperature` readings land on the same partition.
- All `humidity` readings land on the same partition.
- Offsets within each partition are strictly increasing.
- Temperature values above 35°C trigger the alert message.

5.5 Step 4 – Consumer Group Scaling Exercise

5.5.1 4a – Start a Second Consumer Instance

Without stopping the first consumer, open a **third terminal** and run:

```
# Terminal 3 (same consumer.py, same group_id)
python consumer.py
```

Within a few seconds, Kafka will trigger a **rebalance**. Both consumer instances belong to the group `sensor-analytics`, so Kafka will redistribute the 3 partitions between the 2 consumers (e.g., Consumer 1 gets P0+P1, Consumer 2 gets P2). You will see each terminal only printing messages from its assigned partitions.

5.5.2 4b – Check Partition Assignment

Add this diagnostic block to `consumer.py` (after creating the consumer object, before the loop) to print the assignment:

```
# Force a poll to trigger partition assignment
consumer.poll(timeout_ms=1000)
assigned = consumer.assignment()
print(f"Assigned partitions: "
      f"{[[f'P{p.partition}' for p in assigned]]}")
```

`consumer.assignment()` returns the set of `TopicPartition` objects assigned to this consumer instance. After the rebalance, each terminal should show a different non-overlapping set of partitions. This is the foundation of horizontal scaling: add more consumer instances (up to the number of partitions) to increase read throughput linearly.

5.5.3 4c – Observe the Rebalance on Consumer Exit

Stop one consumer with `Ctrl+C` and watch the other terminal. After `session.timeout.ms` (45 seconds), the Group Coordinator detects the departure and triggers a new rebalance. The remaining consumer will be assigned *all three* partitions.

5.6 Step 5 – Fault Injection: Crash and Replay

This exercise demonstrates the value of manual offset commit.

5.6.1 5a – Produce More Messages

```
# Produce 20 more messages
python producer.py
```

5.6.2 5b – Crash the Consumer Mid-Processing

Start the consumer, let it process a few messages, then press **Ctrl+C** immediately before it can commit:

```
python consumer.py
# Let it print 3-4 messages, then press Ctrl+C quickly
```

Because we are using **manual commit after processing**, pressing Ctrl+C before `consumer.commit()` means those last messages were processed but their offsets were *not* saved. Kafka still considers them unread.

5.6.3 5c – Restart and Observe Replay

```
python consumer.py
```

On restart, the consumer fetches its last committed offset from the `__consumer_offsets` topic and resumes from there. You will see the same messages that were processed just before the crash *reappear*. This is **at-least-once delivery** in action.

Key insight: to avoid undesired side effects on replay (e.g., inserting duplicate rows into a database), your business logic should be *idempotent* – the same message processed twice should produce the same result as processing it once.

5.7 Step 6 – Monitor Consumer Lag in Kafka UI

Open <http://localhost:8080> and navigate to **Consumer Groups** → **sensor-analytics**.

lightbg Column	What it tells you
LAG	Messages produced but not yet consumed. Healthy = 0 or near 0.
OFFSET	Last committed offset for this partition in this group.
END OFFSET	Latest message offset in the partition (high-water mark).
CONSUMER ID	Which consumer instance owns this partition right now.

Experiment: Induce Lag

1. Stop all consumers.
2. Run the producer to send 100 messages.
3. Check Kafka UI – lag should equal 100 for each partition.
4. Restart the consumer and watch the lag drop to 0 in real time.

This demonstrates how consumer lag is a key operational metric: a rising lag indicates the consumer is slower than the producer and may need scaling.

 Reflection Questions

Answer these questions before Session 3:

1. You used `key=sensor_type` in the producer. What would happen to message ordering if you had used `key=None` instead?
2. With `enable.auto.commit=False`, what are the exact conditions under which a message could be processed *twice*? What about *zero* times?
3. You have a topic with 6 partitions and a consumer group with 8 consumers. How many consumers are idle? What is the maximum number of consumers that can usefully process this topic in parallel?
4. Explain the difference between `auto.offset.reset=earliest` and `auto.offset.reset=latest`. In which situations would you choose each?
5. Your producer's throughput is 10 000 messages/second but the broker is only acknowledging 2 000/second. Which producer parameters would you adjust to improve throughput while keeping `acks=all`?

6. Summary & Key Takeaways

📌 What We Covered Today

1. A Kafka partition is an **append-only log**. Messages are immutable, ordered by offset, and retained for a configurable period.
2. The **message key** determines partition routing: same key $\Rightarrow$ same partition $\Rightarrow$ guaranteed ordering for that entity.
3. **Producer durability** is controlled by `acks`, `retries`, and `enable.idempotence`. Use `acks=all` + idempotence for exactly-once writes.
4. **Consumer offset tracking** determines delivery semantics: commit *after* processing for at-least-once; design for idempotency to handle replays safely.
5. **Consumer groups** enable horizontal scaling: Kafka assigns one partition per consumer instance within a group.
6. A **rebalance** redistributes partitions when group membership changes; tune `session.timeout.ms` to balance failure detection speed against unnecessary rebalances.
7. In the lab, we built a Python producer–consumer pipeline, scaled consumption horizontally, simulated a crash, and observed at-least-once replay.

6.1 Vocabulary Checklist

- | | |
|--|---|
| ✓ Append-only log | ✓ Manual offset commit |
| ✓ Offset (consumer vs. partition) | ✓ At-most-once / at-least-once / exactly-once |
| ✓ Message key / partition routing | ✓ Consumer group / group.id |
| ✓ <code>acks</code> (0, 1, all) | ✓ Rebalance / Group Coordinator |
| ✓ Idempotent producer / PID | ✓ ISR / <code>min.insync.replicas</code> |
| ✓ <code>batch.size</code> / <code>linger.ms</code> | ✓ Consumer lag |
| ✓ <code>enable.auto.commit</code> | ✓ <code>auto.offset.reset</code> |

6.2 Further Reading

- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly. Chapter 11 (Stream Processing).
- kafka-python-ng documentation: kafka-python-ng.readthedocs.io

- Confluent – Kafka Producers Deep Dive: developer.confluent.io
- Apache Kafka – Consumer Group Protocol: kafka.apache.org/documentation
- Narkhede, N., Shapira, G., Palino, T. (2021). *Kafka: The Definitive Guide*, 2nd ed. O'Reilly. Chapters 3–4.

6.3 Preview: Session 3

Next session we move from raw messaging to structured data pipelines:

- Batch vs. streaming ETL: concepts and trade-offs.
- Integration with relational databases and cloud storage.
- Introduction to Apache Spark and Spark Streaming.
- Lab: Design a complete ETL pipeline using Kafka + Spark.

Appendix A – Complete producer.py

```
from kafka import KafkaProducer
import json, time, random

producer = KafkaProducer(
    bootstrap_servers=['localhost:9092',
                      'localhost:9094',
                      'localhost:9096'],
    acks='all',
    retries=5,
    max_in_flight_requests_per_connection=1,
    value_serializer=lambda v: json.dumps(v).encode('utf-8'),
    key_serializer=lambda k: k.encode('utf-8'),
    linger_ms=10,
    batch_size=32768,
)

SENSOR_TYPES = ['temperature', 'humidity', 'pressure']
TOPIC = 'sensor-events'

try:
    for i in range(50):
        sensor = random.choice(SENSOR_TYPES)
        value = round(random.uniform(10, 40), 2)
        record = {
            'sensor': sensor,
            'value': value,
            'unit': {'temperature': 'C',
                    'humidity': '%',
                    'pressure': 'hPa'}[sensor],
            'timestamp': int(time.time() * 1000),
        }
        meta = producer.send(
            topic=TOPIC,
            key=sensor,
            value=record,
        ).get(timeout=10)
        print(f"[P{meta.partition}]0{meta.offset}] "
              f"{sensor}={value}")
        time.sleep(0.2)
finally:
    producer.flush()
    producer.close()
    print("Done.")
```

Appendix B – Complete consumer.py

```
from kafka import KafkaConsumer
import json

consumer = KafkaConsumer(
    'sensor-events',
    bootstrap_servers=['localhost:9092',
                      'localhost:9094',
                      'localhost:9096'],
    group_id='sensor-analytics',
    auto_offset_reset='earliest',
    enable_auto_commit=False,
    value_deserializer=lambda b: json.loads(b.decode('utf-8')),
    key_deserializer=lambda b: b.decode('utf-8') if b else None,
    max_poll_records=10,
    session_timeout_ms=45000,
    heartbeat_interval_ms=15000,
)

# Print assigned partitions after first poll
consumer.poll(timeout_ms=1000)
assigned = consumer.assignment()
print(f"Assigned: {[f'P{p.partition}' for p in assigned]}")

print("Consumer started. Ctrl+C to stop.")
try:
    for message in consumer:
        record = message.value
        key     = message.key
        part    = message.partition
        offset  = message.offset

        print(f"[P{part}|O{offset}] "
              f"sensor={key} "
              f"value={record['value']} {record['unit']}")

        if record['sensor'] == 'temperature' \
            and record['value'] > 35:
            print(f"  >>> ALERT: high temp "
                  f"{record['value']} C !")

        consumer.commit()

except KeyboardInterrupt:
    print("Stopping...")
finally:
    consumer.close()
    print("Consumer closed.")
```

Appendix C – Kafka CLI Quick Reference

```
#           Topics

kafka-topics --bootstrap-server kafka1:29092 --list
kafka-topics --bootstrap-server kafka1:29092 \
  --create --topic <name> --partitions 3 --replication-factor 3
kafka-topics --bootstrap-server kafka1:29092 \
  --describe --topic <name>

#           Producer (CLI)

kafka-console-producer \
  --bootstrap-server kafka1:29092 --topic <name>

#           Consumer (CLI)

kafka-console-consumer \
  --bootstrap-server kafka1:29092 --topic <name> \
  --group mygroup --from-beginning

#           Consumer Group Lag

kafka-consumer-groups \
  --bootstrap-server kafka1:29092 \
  --describe --group <group-id>

#           Reset Offsets (replay from beginning)

kafka-consumer-groups \
  --bootstrap-server kafka1:29092 \
  --group <group-id> --topic <name> \
  --reset-offsets --to-earliest --execute

#           Reset Offsets (skip to latest)

kafka-consumer-groups \
  --bootstrap-server kafka1:29092 \
  --group <group-id> --topic <name> \
  --reset-offsets --to-latest --execute
```

Appendix D – Glossary

lightbg Term	Definition
acks	Producer setting: how many broker acks to wait for before a send is considered successful.
At-least-once	Delivery guarantee: messages are never lost but may be processed more than once.
Auto-commit	Consumer mode: offsets committed automatically in background at a fixed interval.
Batch size	Max bytes accumulated in a producer buffer before sending to the broker.
Consumer group	Named set of consumers that cooperatively read partitions of a topic.
Consumer lag	Number of messages produced but not yet committed by a consumer group.
Exactly-once	Delivery guarantee: no message is lost and none is processed more than once.
Group Coordinator	Broker responsible for managing consumer group membership and offset commits.
Heartbeat	Signal sent by consumer to the Group Coordinator to indicate it is alive.
Idempotent producer	Producer that assigns sequence numbers to prevent broker-side duplicates.
ISR	In-Sync Replica: follower fully caught up with partition leader.
linger.ms	Max time producer waits before sending a non-full batch.
Manual commit	Consumer mode: application explicitly calls <code>commit()</code> after processing.
Offset	Unique, sequential integer identifying a record's position in a partition.
Rebalance	Redistribution of partition assignments when consumer group membership changes.
Serialiser	Function that converts a Python object to bytes for Kafka transport.